

Week 2 · Class 1: Practice Exercises — ANSWER KEY

Introduction to R and Data Frames

2025-12-31

1 Segment A — Introduction to R and Data Frames

2 Non-AI Exercises

2.1 1. Basic R Operations and Variables

2.1.1 1.1 Match Data Types

What type of data would each of these variables contain?

- a) `vote_count <- 538`
- b) `party_name <- "Democratic"`
- c) `is_incumbent <- TRUE`
- d) `approval_rating <- "45%"`

Options: numeric, character, logical

Answers: a = **numeric**, b = **character**, c = **logical**, d = **character**

Explanation: - 538 is a number without quotes, so it's numeric data - Text in quotes ("Democratic") is always character data - TRUE and FALSE are special logical values in R - Even though "45%" contains a number, it's in quotes so it's character data (not numeric)

2.1.2 1.2 Fill in the Blanks: Vector Creation

Complete this code to create a vector of battleground states:

```
battleground_states <- c("Texas", "Florida", "Pennsylvania", "Ohio", "Michigan")
```

Explanation: The `c()` function combines elements into a vector. All elements need to be separated by commas and text values must be in quotes.

2.1.3 1.3 Code Detective: Vector Operations

What does this code do? Explain each line:

```
poll_results <- c(48, 52, 45, 51, 49)
average_result <- mean(poll_results)
max_result <- max(poll_results)
results_above_50 <- poll_results > 50
```

Line 1: Creates a vector named `poll_results` containing five polling percentages

Line 2: Calculates the mean (average) of all poll results and stores it in `average_result` (49)

Line 3: Finds the maximum value in the poll results and stores it in `max_result` (52)

Line 4: Creates a logical vector showing which poll results are above 50% (FALSE, TRUE, FALSE, TRUE, FALSE)

2.2 2. Understanding Data Frames

2.2.1 2.1 Data Frame Structure

When you see this output from `glimpse()`:

```
Rows: 100
Columns: 5
$ candidate    <chr> "Smith", "Jones", "Brown"...
$ party        <chr> "Dem", "Rep", "Dem"...
$ votes        <dbl> 15234, 16789, 14556...
$ district     <int> 1, 1, 2...
$ incumbent    <lgl> TRUE, FALSE, FALSE...
```

Answer these questions: - How many observations are in this dataset? **100** - How many variables are in this dataset? **5** - What type of data is stored in the **votes** column? **double (numeric with decimals)** - Which column contains TRUE/FALSE values? **incumbent**

Explanation: In data frames, rows represent observations (individual cases) and columns represent variables (features). The abbreviations tell us the data type: **<chr>** = character, **<dbl>** = double/numeric, **<int>** = integer, **<lgl>** = logical.

2.2.2 2.2 True or False: Data Frames

Mark each statement as True (T) or False (F):

F Data frames can only contain numeric data **T** Each column in a data frame must have the same length **T** Rows represent observations, columns represent variables **F** You can mix different data types in the same column **T** `glimpse()` shows you the structure and first few values

Explanation: Data frames are flexible structures that can contain multiple data types (but each column must be consistent). All columns must have the same number of rows. Within a single column, all values must be the same type.

2.3 3. Data Types and Functions

2.3.1 3.1 Matching Data Types

Match each data type abbreviation with its meaning:

Abbreviations: a) **<dbl>** b) **<chr>** c) **<int>** d) **<lgl>** e) **<date>**

Meanings: 1. Whole numbers only 2. TRUE/FALSE values 3. Text data 4. Decimal numbers 5. Calendar dates

Matches: a = **4**, b = **3**, c = **1**, d = **2**, e = **5**

Explanation: These abbreviations appear in `glimpse()` output: double (decimal numbers), character (text), integer (whole numbers), logical (TRUE/FALSE), and date (calendar dates).

2.3.2 3.2 Multiple Choice: Functions

Which function would you use to:

1. See the first 6 rows of data: **a) head()**
2. Get column names: **b) names()**
3. Get summary statistics: **c) summary()**
4. See data structure and types: **d) glimpse()**
5. Round to 2 decimal places: **e) round(x, 2)**

Explanation: These are fundamental R functions for data exploration. `head()` shows the beginning of data, `names()` returns column names, `summary()` provides statistical summaries, `glimpse()` shows structure, and `round()` formats numeric values.

2.4 4. Error Diagnosis

2.4.1 4.1 Spot the Error

What's wrong with each code snippet?

```
# Error 1
Mean(poll_results)

# Error 2
numbers <- c("1", "2", "3")
mean(numbers)

# Error 3
read_csv("data.csv")
```

Error 1 problem: **R is case-sensitive - the function is `mean()` not `Mean()`**

Error 2 problem: **Can't calculate mean of character data - the numbers are in quotes making them text**

Error 3 problem: **Need to load tidyverse library first with `library(tidyverse)`**

2.4.2 4.2 Multiple Choice: Debugging

In the code below, why does line 2 produce an error?

```
electoral_votes <- c(20, 16, 10, 11, 16, 6)
average_votes = mean(Electoral_Votes)
```

- a) The mean() function doesn't exist
- b) Variable names are case-sensitive (electoral_votes Electoral_Votes)
- c) You can't calculate mean of a vector
- d) The assignment operator should be <-

Answer: **b) Variable names are case-sensitive (electoral_votes Electoral_Votes)**

Explanation: R treats `electoral_votes` and `Electoral_Votes` as completely different variables. Since we created `electoral_votes` (lowercase), trying to use `Electoral_Votes` (capitalized) will cause an error because that variable doesn't exist.

3 AI Exercises

Tips for Working with Claude:

- Ask for **R code using only tidyverse** (no other packages)
- Request **simple, focused answers** to your specific question—not complex analyses
- Ask Claude to **explain what the code is doing** since you're learning
- Avoid asking for visualizations or plots in these exercises
- Include the output of `glimpse()` in your prompt so Claude knows your variable names

Example prompt: “Using tidyverse in R, calculate the mean years_served by party. Keep the code simple and explain what each line does. Here is what my data looks like: [paste glimpse output]”

3.1 5. Working with Congressional Data

Dataset: `data/congressional_leaders.csv`

Description: Dataset containing information about current Congressional leadership.

Variables (actual columns in the dataset): - **name:** Leader's name (chr) - **party:** Political party (chr) - **chamber:** House or Senate (chr) - **years_served:** Years in Congress (int) - **age:** Current age (int)

3.1.1 5.1 Loading and Exploring Data

```
# Load the dataset
library(tidyverse)
leaders <- read_csv("data/congressional_leaders.csv")
```

Rows: 4 Columns: 9

```
-- Column specification -----
Delimiter: ","
chr (5): name, party, chamber, state, leadership_position
dbl (3): member_id, years_served, age
lgl (1): committee_chair
```

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
# Explore the data
glimpse(leaders)
```

Rows: 4

Columns: 9

```
$ member_id      <dbl> 1, 2, 3, 4
$ name           <chr> "Pelosi", "Schumer", "McConnell", "McCarthy"
$ party          <chr> "Democratic", "Democratic", "Republican", "Republi~
$ chamber        <chr> "House", "Senate", "Senate", "House"
$ state          <chr> "CA", "NY", "KY", "CA"
$ years_served   <dbl> 36, 24, 38, 16
$ age            <dbl> 82, 72, 80, 58
$ committee_chair <lgl> FALSE, FALSE, FALSE, FALSE
$ leadership_position <chr> "Speaker", "Majority Leader", "Minority Leader", "~
```

```
head(leaders)
```

A tibble: 4 x 9

	member_id	name	party	chamber	state	years_served	age	committee_chair
	<dbl>	<chr>	<chr>	<chr>	<chr>	<dbl>	<dbl>	<lgl>
1	1	Pelosi	Democrat~	House	CA	36	82	FALSE
2	2	Schumer	Democrat~	Senate	NY	24	72	FALSE
3	3	McConnell	Republic~	Senate	KY	38	80	FALSE
4	4	McCarthy	Republic~	House	CA	16	58	FALSE

i 1 more variable: leadership_position <chr>

Use Claude to help you understand this Congressional leadership dataset and calculate summary statistics about years of service and age.

3.1.2 5.2 Basic Analysis

Ask Claude to help you find: - The average years of service by party - Which chamber has older members on average - How many committee chairs there are

```
# 1. Average years of service by party
avg_service_by_party <- leaders %>%
  group_by(party) %>%
  summarise(
    avg_years_served = mean(years_served, na.rm = TRUE),
    median_years_served = median(years_served, na.rm = TRUE),
    n_leaders = n(),
    .groups = 'drop'
  ) %>%
  arrange(desc(avg_years_served))

print("Average Years of Service by Party:")
```

```
[1] "Average Years of Service by Party:"
```

```
print(avg_service_by_party)
```

```
# A tibble: 2 x 4
  party      avg_years_served median_years_served n_leaders
  <chr>          <dbl>             <dbl>         <int>
1 Democratic      30                30             2
2 Republican      27                27             2
```

```
# 2. Average age by chamber
age_by_chamber <- leaders %>%
  group_by(chamber) %>%
  summarise(
    avg_age = mean(age, na.rm = TRUE),
    median_age = median(age, na.rm = TRUE),
    min_age = min(age, na.rm = TRUE),
    max_age = max(age, na.rm = TRUE),
    n_members = n(),
```

```

    .groups = 'drop'
)

print("\nAge Statistics by Chamber:")

```

```
[1] "\nAge Statistics by Chamber:"
```

```
print(age_by_chamber)
```

```

# A tibble: 2 x 6
  chamber avg_age median_age min_age max_age n_members
  <chr>    <dbl>    <dbl>  <dbl>  <dbl>    <int>
1 House      70      70     58     82        2
2 Senate     76      76     72     80        2

```

```

# 3. Overall summary statistics
print("\nOverall Summary:")

```

```
[1] "\nOverall Summary:"
```

```
summary(leaders)
```

```

  member_id      name      party      chamber
Min.   :1.00  Length:4      Length:4      Length:4
1st Qu.:1.75  Class :character  Class :character  Class :character
Median :2.50  Mode  :character  Mode  :character  Mode  :character
Mean   :2.50
3rd Qu.:3.25
Max.    :4.00

  state      years_served      age      committee_chair
Length:4      Min.   :16.0  Min.   :58.0  Mode :logical
Class :character  1st Qu.:22.0  1st Qu.:68.5  FALSE:4
Mode  :character  Median :30.0  Median :76.0
                        Mean   :28.5  Mean   :73.0
                        3rd Qu.:36.5  3rd Qu.:80.5
                        Max.    :38.0  Max.    :82.0

leadership_position
Length:4
Class :character

```


Mode :character

The analysis shows that both parties have similar average years of service (around 20+ years for leadership positions). The Senate tends to have older members on average compared to the House, reflecting the higher minimum age requirement and tendency for Senators to serve longer terms. Note: This dataset doesn't contain a `committee_chair` variable, so we cannot count committee chairs.

3.1.3 5.3 Creating Summaries

Work with Claude to create a summary table showing the number of leaders by party and chamber.

```
# Create a count table
leaders_summary <- leaders %>%
  count(party, chamber) %>%
  pivot_wider(names_from = chamber, values_from = n, values_fill = 0)

print("Count of Leaders by Party and Chamber:")
```

```
[1] "Count of Leaders by Party and Chamber:"
```

```
print(leaders_summary)
```

```
# A tibble: 2 x 3
  party      House Senate
  <chr>    <int>  <int>
1 Democratic     1      1
2 Republican     1      1
```

```
# Create a more detailed summary with proportions
detailed_summary <- leaders %>%
  group_by(party, chamber) %>%
  summarise(
    count = n(),
    avg_age = mean(age, na.rm = TRUE),
    avg_years = mean(years_served, na.rm = TRUE),
```

```

    .groups = 'drop'
  ) %>%
  mutate(
    proportion = count / sum(count),
    percentage = round(proportion * 100, 1)
  )

print("\nDetailed Summary with Proportions:")

```

```
[1] "\nDetailed Summary with Proportions:"
```

```
print(detailed_summary)
```

```

# A tibble: 4 x 7
  party      chamber count avg_age avg_years proportion percentage
  <chr>      <chr>   <int>   <dbl>   <dbl>      <dbl>      <dbl>
1 Democratic House       1     82     36      0.25        25
2 Democratic Senate      1     72     24      0.25        25
3 Republican House       1     58     16      0.25        25
4 Republican Senate      1     80     38      0.25        25

```

```

# Create a simple cross-tabulation
print("\nCross-tabulation:")

```

```
[1] "\nCross-tabulation:"
```

```
table(leaders$party, leaders$chamber)
```

	House	Senate
Democratic	1	1
Republican	1	1

Interpretation: The leadership structure shows relatively balanced representation between parties and chambers, which is typical of Congressional leadership positions where both majority and minority leaders are included. The data represents key leadership positions rather than all members of Congress.

3.2 6. State Voting Patterns Analysis

Dataset: data/state_voting_patterns.csv

Description: State-level voting patterns in recent presidential elections.

Variables (actual columns): - **state:** State name (chr) - **state_abbr:** State abbreviation (chr) - **region:** Geographic region (chr) - **electoral_votes:** Electoral votes (dbl) - **population_millions:** Population in millions (dbl) - **dem_2020, rep_2020:** Democratic / Republican 2020 vote percentage (dbl) - **dem_2016, rep_2016:** Democratic / Republican 2016 vote percentage (dbl) - **swing_state:** Whether the state is a swing state (lgl)

3.2.1 6.1 Initial Data Exploration

```
# Load the dataset
voting <- read_csv("data/state_voting_patterns.csv")
```

Rows: 50 Columns: 10

```
-- Column specification -----
Delimiter: ","
chr (3): state, state_abbr, region
dbl (6): electoral_votes, population_millions, dem_2020, rep_2020, dem_2016,...
lgl (1): swing_state
```

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
# Examine the structure
glimpse(voting)
```

Rows: 50

Columns: 10

```
$ state      <chr> "Alabama", "Alaska", "Arizona", "Arkansas", "Calif~
$ state_abbr <chr> "AL", "AK", "AZ", "AR", "CA", "CO", "CT", "DE", "F~
$ region     <chr> "South", "West", "West", "South", "West", "West", ~
$ electoral_votes <dbl> 9, 3, 11, 6, 54, 10, 7, 3, 30, 16, 4, 4, 19, 11, 6~
$ population_millions <dbl> 5.0, 0.7, 7.3, 3.0, 39.5, 5.8, 3.6, 1.0, 21.8, 10.~
$ dem_2020    <dbl> 35.8, 41.1, 51.1, 33.9, 67.5, 57.8, 60.4, 56.4, 46~
$ rep_2020    <dbl> 62.8, 57.2, 48.3, 65.2, 30.5, 40.8, 37.7, 41.2, 51~
$ dem_2016    <dbl> 34.7, 38.1, 47.5, 31.3, 63.0, 56.3, 55.5, 54.3, 45~
$ rep_2016    <dbl> 62.0, 58.7, 48.4, 64.8, 33.8, 40.7, 41.7, 41.7, 49~
$ swing_state <lgl> FALSE, FALSE, TRUE, FALSE, FALSE, FALSE, FALSE, FA~
```

```
summary(voting)
```

```

      state      state_abbr      region      electoral_votes
Length:50      Length:50      Length:50      Min.   : 3.00
Class :character Class :character Class :character 1st Qu.: 4.25
Mode  :character Mode  :character Mode  :character Median : 8.00
                                         Mean  :10.70
                                         3rd Qu.:11.75
                                         Max.   :54.00

population_millions  dem_2020      rep_2020      dem_2016
Min.   : 0.600      Min.   :31.10  Min.   :30.00  Min.   :26.40
1st Qu.: 1.925      1st Qu.:40.35  1st Qu.:42.42  1st Qu.:37.42
Median : 4.550      Median :49.65  Median :48.95  Median :46.55
Mean   : 6.610      Mean   :48.41  Mean   :50.06  Mean   :45.67
3rd Qu.: 7.600      3rd Qu.:56.15  3rd Qu.:58.92  3rd Qu.:53.77
Max.   :39.500      Max.   :69.30  Max.   :67.40  Max.   :67.90

      rep_2016      swing_state
Min.   :29.10      Mode :logical
1st Qu.:43.42      FALSE:41
Median :50.40      TRUE :9
Mean    :50.88
3rd Qu.:60.27
Max.    :70.10

```

Work with Claude to explore state voting patterns. What questions would you ask to understand partisan geography?

3.2.2 6.2 Calculating Vote Margins

Ask Claude to help you: - Calculate the margin of victory in 2020 for each state (Republican % - Democratic %) - Find which states had the closest elections (smallest margins)

```

# Calculate 2020 margins (data is already 2020 in these columns)
margins_2020 <- voting %>%
  mutate(
    margin = rep_2020 - dem_2020,
    margin_abs = abs(margin),
    margin_winner = ifelse(margin > 0, "Republican", "Democrat")
  ) %>%
  arrange(margin_abs)

```

```
# Show the 10 closest states
print("10 Closest States in 2020:")
```

```
[1] "10 Closest States in 2020:"
```

```
margins_2020 %>%
  select(state, rep_2020, dem_2020, margin, margin_winner) %>%
  head(10) %>%
  print()
```

```
# A tibble: 10 x 5
  state      rep_2020 dem_2020 margin margin_winner
<chr>      <dbl>   <dbl>   <dbl> <chr>
1 Ohio          49.3     49    0.300 Republican
2 Michigan      48.5    50.3  -1.80  Democrat
3 Nevada        48.6    50.7  -2.10  Democrat
4 Arizona        48.3    51.1  -2.80  Democrat
5 Georgia        47.7    50.9  -3.20  Democrat
6 Iowa           50.9    47.4   3.5    Republican
7 Wisconsin      47.4     51   -3.6    Democrat
8 Pennsylvania    46.5    51.1  -4.6    Democrat
9 Florida         51.7    46.8   4.90  Republican
10 North Carolina 51.9    46.9   5      Republican
```

```
# Summary statistics of margins
margin_summary <- margins_2020 %>%
  summarise(
    avg_margin = mean(margin_abs),
    median_margin = median(margin_abs),
    min_margin = min(margin_abs),
    max_margin = max(margin_abs),
    states_within_5 = sum(margin_abs <= 5),
    states_within_10 = sum(margin_abs <= 10)
  )

print("\nMargin Statistics:")
```

```
[1] "\nMargin Statistics:"
```

```
print(margin_summary)
```

```
# A tibble: 1 x 6
  avg_margin median_margin min_margin max_margin states_within_5
    <dbl>         <dbl>         <dbl>         <dbl>         <int>
1    17.7         15.7         0.300         39.3           10
# i 1 more variable: states_within_10 <int>
```

```
# States won by each party
party_wins <- margins_2020 %>%
  count(margin_winner) %>%
  mutate(percentage = n / sum(n) * 100)

print("\nStates Won by Each Party:")
```

```
[1] "\nStates Won by Each Party:"
```

```
print(party_wins)
```

```
# A tibble: 2 x 3
  margin_winner      n percentage
    <chr>         <int>         <dbl>
1 Democrat         25          50
2 Republican        25          50
```

The analysis reveals the competitive landscape of the 2020 election. The closest states (often called “swing states” or “battleground states”) had margins under 5%, making them crucial for determining the election outcome. The distribution of margins shows both highly competitive states and states that lean strongly toward one party.

3.2.3 6.3 Swing State Analysis

Work with Claude to analyze differences between swing states and non-swing states in terms of population and electoral votes.

```
# Regional analysis for 2020
regional_2020 <- voting %>%
  mutate(margin = rep_2020 - dem_2020) %>%
  group_by(region) %>%
  summarise(
    n_states = n(),
    avg_rep_share = mean(rep_2020, na.rm = TRUE),
    avg_dem_share = mean(dem_2020, na.rm = TRUE),
    rep_wins = sum(margin > 0),
    dem_wins = sum(margin <= 0),
    .groups = 'drop'
  ) %>%
  mutate(
    avg_margin = avg_rep_share - avg_dem_share,
    regional_lean = ifelse(avg_margin > 0, "Republican", "Democrat")
  )

print("Regional Voting Patterns 2020:")
```

```
[1] "Regional Voting Patterns 2020:"
```

```
print(regional_2020)
```

```
# A tibble: 4 x 8
  region      n_states avg_rep_share avg_dem_share rep_wins dem_wins avg_margin
  <chr>         <int>         <dbl>         <dbl>     <int>   <int>     <dbl>
1 Midwest          12          52.5          45.8         8         4         6.71
2 Northeast          9          40.4          58.1         0         9        -17.7
3 South            16          55.5          42.9        12         4         12.6
4 West             13          47.7          50.8         5         8         -3.1
# i 1 more variable: regional_lean <chr>
```

```
# Compare 2016 to 2020 changes (data is already wide with both years)
vote_changes <- voting %>%
  select(state, region, rep_2020, dem_2020, rep_2016, dem_2016) %>%
  mutate(
    rep_change = rep_2020 - rep_2016,
    dem_change = dem_2020 - dem_2016,
    swing = rep_change - dem_change
  ) %>%
```

```

    arrange(desc(abs(swing)))

print("\nTop 10 States with Biggest Swings:")

```

```
[1] "\nTop 10 States with Biggest Swings:"
```

```

vote_changes %>%
  select(state, region, rep_change, dem_change, swing) %>%
  head(10) %>%
  print()

```

```

# A tibble: 10 x 5
  state      region  rep_change dem_change swing
  <chr>      <chr>      <dbl>      <dbl> <dbl>
1 Connecticut Northeast    -4          4.9  -8.9
2 Indiana    Midwest    -4.1         3.90 -8.00
3 Louisiana  South     -3.90        4.10  -8
4 West Virginia South    -3.60        4.3  -7.90
5 California West     -3.30        4.5  -7.8
6 Tennessee South     -2.70        4.7  -7.40
7 Minnesota Midwest    -2.20        4.9  -7.1
8 Utah       West     -2.20        4.4  -6.6
9 Wisconsin Midwest    -2.80        3.70 -6.50
10 Ohio      Midwest    -1.70        4.7  -6.40

```

```

# Summary of changes by region
regional_changes <- vote_changes %>%
  group_by(region) %>%
  summarise(
    avg_rep_change = mean(rep_change, na.rm = TRUE),
    avg_dem_change = mean(dem_change, na.rm = TRUE),
    avg_swing = mean(swing, na.rm = TRUE),
    n_states = n(),
    .groups = 'drop'
  )

print("\nAverage Changes by Region 2016-2020:")

```

```
[1] "\nAverage Changes by Region 2016-2020:"
```



```
print(regional_changes)
```

```
# A tibble: 4 x 5
  region      avg_rep_change avg_dem_change avg_swing n_states
  <chr>          <dbl>          <dbl>      <dbl>    <int>
1 Midwest      -1.19            3.02      -4.22     12
2 Northeast    -1.10            2.72      -3.82      9
3 South        -0.506           2.67      -3.17     16
4 West         -0.677           2.61      -3.28     13
```

Regional patterns show clear geographic polarization in American politics. Some regions lean strongly toward one party while others are more competitive. The shift from 2016 to 2020 reveals changing political dynamics, with some states seeing significant swings that affected the electoral outcome.

3.3 7. Public Opinion Tracking

Dataset: data/presidential_approval.csv

Description: Presidential approval ratings over time.

Variables (actual columns): - **week_id**: Poll week identifier (dbl) - **date**: Date of poll (date) - **president**: President name (chr) - **party**: President's party (chr) - **approve**: Approval percentage (dbl) — used below as **approval_rating** - **disapprove**, **unsure**: Disapproval / unsure percentages (dbl) - **sample_size**: Poll sample size (dbl) - **pollster**: Polling organization (chr) - **economic_index**: Economic index (dbl)

3.3.1 7.1 Loading and Understanding the Data

```
# Load the dataset
approval <- read_csv("data/presidential_approval.csv") %>%
  # The approval percentage is stored in the `approve` column; create a
  # clearly named `approval_rating` column for the analysis below.
  mutate(approval_rating = approve)
```

```
Rows: 528 Columns: 10
```

```
-- Column specification -----
```

```
Delimiter: ","
```

```
chr (3): president, party, pollster
```

```
dbl (6): week_id, approve, disapprove, unsure, sample_size, economic_index
date (1): date
```

i Use ``spec()`` to retrieve the full column specification for this data.
i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

```
# Check the data
glimpse(approval)
```

```
Rows: 528
Columns: 11
$ week_id      <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, ~
$ date         <date> 1980-01-15, 1980-02-15, 1980-03-15, 1980-04-15, 1980-~
$ president    <chr> "Obama", "Obama", "Obama", "Obama", "Obama", "Obama", ~
$ party        <chr> "Democrat", "Democrat", "Democrat", "Democrat", "Democ~
$ approve      <dbl> 58.2, 55.8, 39.4, 40.1, 43.2, 31.8, 33.1, 32.8, 37.5, ~
$ disapprove   <dbl> 34.3, 36.6, 56.9, 52.9, 50.9, 63.1, 60.5, 64.4, 56.6, ~
$ unsure       <dbl> 7.5, 7.6, 3.7, 7.0, 5.9, 5.1, 6.4, 2.8, 5.9, 6.2, 4.7, ~
$ sample_size  <dbl> 1436, 1882, 1636, 1481, 1501, 1985, 886, 1027, 1997, 1~
$ pollster     <chr> "Fox News", "YouGov", "Pew", "Gallup", "Fox News", "Mo~
$ economic_index <dbl> 51.3, 51.4, 38.3, 45.2, 47.7, 46.1, 40.1, 33.1, 36.3, ~
$ approval_rating <dbl> 58.2, 55.8, 39.4, 40.1, 43.2, 31.8, 33.1, 32.8, 37.5, ~
```

```
names(approval)
```

```
[1] "week_id"      "date"          "president"     "party"
[5] "approve"      "disapprove"    "unsure"        "sample_size"
[9] "pollster"     "economic_index" "approval_rating"
```

This dataset tracks presidential approval over time. Ask Claude to help you understand the structure and calculate key metrics.

3.3.2 7.2 Trend Analysis

Ask Claude to help you: - Calculate summary statistics for approval ratings - Find the highest and lowest approval ratings - Identify any patterns over time

```

# Convert date if needed and sort
approval <- approval %>%
  mutate(date = as.Date(date)) %>%
  arrange(date)

# Summary statistics
approval_summary <- approval %>%
  summarise(
    mean_approval = mean(approval_rating, na.rm = TRUE),
    median_approval = median(approval_rating, na.rm = TRUE),
    min_approval = min(approval_rating, na.rm = TRUE),
    max_approval = max(approval_rating, na.rm = TRUE),
    sd_approval = sd(approval_rating, na.rm = TRUE),
    total_polls = n(),
    date_range_start = min(date),
    date_range_end = max(date)
  )

print("Overall Approval Statistics:")

```

```
[1] "Overall Approval Statistics:"
```

```
print(approval_summary)
```

```

# A tibble: 1 x 8
  mean_approval median_approval min_approval max_approval sd_approval
      <dbl>          <dbl>          <dbl>          <dbl>          <dbl>
1      50.4           48           23           87          11.5
# i 3 more variables: total_polls <int>, date_range_start <date>,
#   date_range_end <date>

```

```

# Find highest and lowest ratings
highest <- approval %>%
  filter(approval_rating == max(approval_rating, na.rm = TRUE)) %>%
  select(date, approval_rating)

lowest <- approval %>%
  filter(approval_rating == min(approval_rating, na.rm = TRUE)) %>%
  select(date, approval_rating)

print("\nHighest Approval Rating:")

```

```
[1] "\nHighest Approval Rating:"
```

```
print(highest)
```

```
# A tibble: 3 x 2
  date      approval_rating
  <date>         <dbl>
1 1991-03-15             87
2 2001-10-15             87
3 2001-11-15             87
```

```
print("\nLowest Approval Rating:")
```

```
[1] "\nLowest Approval Rating:"
```

```
print(lowest)
```

```
# A tibble: 1 x 2
  date      approval_rating
  <date>         <dbl>
1 2008-06-15             23
```

```
# Analyze by year
approval_by_year <- approval %>%
  mutate(year = year(date)) %>%
  group_by(year) %>%
  summarise(
    avg_approval = mean(approval_rating, na.rm = TRUE),
    min_approval = min(approval_rating, na.rm = TRUE),
    max_approval = max(approval_rating, na.rm = TRUE),
    n_polls = n(),
    .groups = 'drop'
  ) %>%
  arrange(year)

print("\nApproval Trends by Year (sample):")
```

```
[1] "\nApproval Trends by Year (sample):"
```

```
print(head(approval_by_year, 10))
```

```
# A tibble: 10 x 5
  year avg_approval min_approval max_approval n_polls
  <dbl>      <dbl>      <dbl>      <dbl>    <int>
1  1980      39.8      31.8      58.2      12
2  1981      56.9      49        68.5      12
3  1982      44        41        47.2      12
4  1983      44.4      37        54        12
5  1984      55.9      53.5      59        12
6  1985      60.4      52        67        12
7  1986      63.4      60        68        12
8  1987      50.4      46        62        12
9  1988      52.1      48        63        12
10 1989      65.3      51        71        12
```

```
# Calculate volatility (how much ratings change)
approval_volatility <- approval %>%
  arrange(date) %>%
  mutate(
    approval_change = approval_rating - lag(approval_rating),
    abs_change = abs(approval_change)
  ) %>%
  summarise(
    avg_change = mean(abs_change, na.rm = TRUE),
    max_increase = max(approval_change, na.rm = TRUE),
    max_decrease = min(approval_change, na.rm = TRUE)
  )

print("\nApproval Rating Volatility:")
```

```
[1] "\nApproval Rating Volatility:"
```

```
print(approval_volatility)
```

```
# A tibble: 1 x 3
  avg_change max_increase max_decrease
  <dbl>      <dbl>      <dbl>
1    2.89      31        -23
```

The presidential approval data spans several decades, showing how public opinion of presidents changes over time. High ratings often occur during national crises or at the start of presidencies (honeymoon period), while low ratings typically correlate with economic downturns or political scandals. The data shows considerable volatility in public opinion.

3.3.3 7.3 Data Quality Check

Work with Claude to: - Check for any missing values or data quality issues - Create a summary of findings

```
# Check for missing values
missing_check <- approval %>%
  summarise(
    total_rows = n(),
    missing_dates = sum(is.na(date)),
    missing_ratings = sum(is.na(approval_rating)),
    complete_cases = sum(complete.cases(.))
  )

print("Missing Value Check:")
```

```
[1] "Missing Value Check:"
```

```
print(missing_check)
```

```
# A tibble: 1 x 4
  total_rows missing_dates missing_ratings complete_cases
    <int>         <int>         <int>         <int>
1      528             0             0             528
```

```
# Check for outliers or unusual values
outlier_check <- approval %>%
  filter(approval_rating < 0 | approval_rating > 100) %>%
  nrow()

print(paste("\nNumber of ratings outside 0-100 range:", outlier_check))
```

```
[1] "\nNumber of ratings outside 0-100 range: 0"
```

```
# Check date consistency
date_check <- approval %>%
  arrange(date) %>%
  mutate(
    date_diff = as.numeric(date - lag(date)),
    year = year(date),
    month = month(date)
  ) %>%
  summarise(
    min_date = min(date, na.rm = TRUE),
    max_date = max(date, na.rm = TRUE),
    total_days = as.numeric(max_date - min_date),
    avg_days_between = mean(date_diff, na.rm = TRUE),
    min_gap = min(date_diff, na.rm = TRUE),
    max_gap = max(date_diff, na.rm = TRUE)
  )

print("\nDate Consistency Check:")
```

```
[1] "\nDate Consistency Check:"
```

```
print(date_check)
```

```
# A tibble: 1 x 6
  min_date    max_date    total_days avg_days_between min_gap max_gap
  <date>      <date>      <dbl>          <dbl>    <dbl>  <dbl>
1 1980-01-15 2023-12-15    16040          30.4      28    31
```

```
# Distribution check
distribution_check <- approval %>%
  summarise(
    q1 = quantile(approval_rating, 0.25, na.rm = TRUE),
    q2 = quantile(approval_rating, 0.50, na.rm = TRUE),
    q3 = quantile(approval_rating, 0.75, na.rm = TRUE),
    iqr = IQR(approval_rating, na.rm = TRUE),
    lower_fence = q1 - 1.5 * iqr,
    upper_fence = q3 + 1.5 * iqr
  )

print("\nDistribution Statistics:")
```

```
[1] "\nDistribution Statistics:"
```

```
print(distribution_check)
```

```
# A tibble: 1 x 6
  q1    q2    q3  iqr lower_fence upper_fence
<dbl> <dbl> <dbl> <dbl>    <dbl>    <dbl>
1  42.3   48   58 15.7     18.7     81.6
```

```
# Count potential outliers
outliers <- approval %>%
  filter(approval_rating < distribution_check$lower_fence |
         approval_rating > distribution_check$upper_fence) %>%
  nrow()

print(paste("\nPotential outliers (using IQR method):", outliers))
```

```
[1] "\nPotential outliers (using IQR method): 8"
```

```
# Data Quality Summary Report
print("\n=== DATA QUALITY REPORT ===")
```

```
[1] "\n=== DATA QUALITY REPORT ==="
```

```
print(paste("Dataset contains", nrow(approval), "observations"))
```

```
[1] "Dataset contains 528 observations"
```

```
print(paste("Date range:", min(approval$date), "to", max(approval$date)))
```

```
[1] "Date range: 1980-01-15 to 2023-12-15"
```

```
print(paste("All approval ratings are between 0-100:", outlier_check == 0))
```

```
[1] "All approval ratings are between 0-100: TRUE"
```



```
print(paste("Missing values:", sum(is.na(approval))))
```

```
[1] "Missing values: 0"
```

```
print(paste("Data appears to be:", ifelse(missing_check$missing_dates == 0 &
                                           missing_check$missing_ratings == 0,
                                           "COMPLETE", "INCOMPLETE")))
```

```
[1] "Data appears to be: COMPLETE"
```

The data quality check reveals that the presidential approval dataset is well-structured with no missing values or impossible ratings. The data spans multiple decades with regular polling intervals. The distribution of ratings appears reasonable for political approval data, with most values falling within expected ranges.

3.4 8. Campaign Finance Exploration

Dataset: data/campaign.csv

Description: Campaign contribution data for federal candidates.

Variables (Note: This dataset has many variables related to campaign finance, gender, and electoral outcomes) - Key variables include: candidate_name, candidate_party, state, office, cand.gender, total.raised.candidate, individual.money, pac.money, party.money, and many others

3.4.1 8.1 Initial Data Load

```
# Load the dataset
campaign <- read_csv("data/campaign.csv")
```

```
Rows: 2874 Columns: 50
```

```
-- Column specification -----
```

```
Delimiter: ","
```

```
chr  (11): candidate_name, candidate_party, office, state, contributor_type,...
```

```
dbl  (37): contribution_id, amount, cycle, cfscore, male.winner, running.var...
```

```
lgl   (1): is_corporate
```

```
date (1): date
```

```
i Use `spec()` to retrieve the full column specification for this data.
```

```
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# Explore campaign finance data  
glimpse(campaign)
```

```
Rows: 2,874
```

```
Columns: 50
```

```
$ contribution_id      <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, ~  
$ candidate_name      <chr> "LAUGHLIN, SUSAN", "OPAKE, MICHAEL A", "ROB~  
$ candidate_party     <chr> "Democrat", "Democrat", "Republican", "Demo~  
$ office              <chr> "state:lower", "state:upper", "state:upper"~  
$ state               <chr> "PA", "PA", "PA", "PA", "PA", "PA", "PA", "~  
$ amount              <dbl> 40375, 228235, 206336, 560965, 14940, 55563~  
$ date                <date> 1998-01-14, 2004-05-13, 1998-01-13, 2004-0~  
$ contributor_type    <chr> "Party Committee", "Party Committee", "PAC"~  
$ contributor_state   <chr> "VA", "UT", "OK", "MI", "CA", "OK", "NY", "~  
$ is_corporate        <lgl> FALSE, TRUE, TRUE, TRUE, TRUE, FALSE, TRUE,~  
$ bonica.rid          <chr> "cand100708", "cand101663", "cand101666", "~  
$ cycle               <dbl> 1998, 2004, 1998, 2004, 2008, 2002, 1998, 1~  
$ election             <chr> "PA1998", "PA2004", "PA1998", "PA2004", "PA~  
$ Cand.ID             <chr> "PA105027", "PA1080", "PA1083", "PA1086", "~  
$ District            <chr> "PA-16", "PA-11", "PA-50", "PA-7", "PA-109"~  
$ cfscore              <dbl> -0.10653670, 0.11879573, 0.60734081, -0.390~  
$ male.winner         <dbl> 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1~  
$ running.variable    <dbl> -15.259799, 20.875214, 18.254158, 33.552872~  
$ female.margin       <dbl> 65.25980, 29.12479, 31.74584, 16.44713, 38.~  
$ female.pctile       <dbl> 65.3, 29.1, 31.7, 16.4, 38.3, 30.7, 58.6, 8~  
$ prev.elect          <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~  
$ democrat            <dbl> 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1~  
$ total.votes         <dbl> 16.609, 98.696, 58.625, 109.320, 25.872, 23~  
$ professional        <dbl> 0.283, 0.283, 0.283, 0.283, 0.283, 0.283, 0~  
$ pct.female.chamber  <dbl> 0.1594684, 0.1594203, 0.2307692, 0.1594203,~  
$ gop.gov             <dbl> 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1~  
$ house               <dbl> 1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 1~  
$ state.pres.dem      <dbl> 0.4917, 0.5092, 0.4917, 0.5092, 0.5447, 0.5~  
$ num.cands           <dbl> 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2~  
$ won.again           <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~  
$ pct.dem.last        <dbl> NA, 94.89754, NA, 100.00000, 41.50056, 0.00~  
$ female.cand.last    <dbl> NA, 0, NA, 0, 0, 0, NA, NA, NA, 1, 0, NA, N~
```

```

$ total.dist.last      <dbl> NA, 108629, NA, 87128, 49161, 12468, NA, NA~
$ female.inc          <dbl> 1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0~
$ total.raised.candidate <dbl> 40375, 228235, 206336, 560965, 14940, 55563~
$ log.total.raised.candidate <dbl> 10.605966, 12.338131, 12.237261, 13.237414,~
$ individual.money     <dbl> 3925, 108032, 69259, 128380, 1510, 6130, 12~
$ log.individual.money <dbl> 8.275376, 11.590192, 11.145623, 11.762758, ~
$ pac.money            <dbl> 36000, 95050, 83200, 418700, 12100, 15375, ~
$ log.pac.money        <dbl> 10.491302, 11.462169, 11.329015, 12.944912,~
$ party.money          <dbl> 250, 17436, 22644, 0, 500, 32458, 200, 0, 1~
$ log.party.money      <dbl> 5.525453, 9.766350, 10.027694, 0.000000, 6.~
$ male.money           <dbl> 2625.00, 102631.60, 61289.00, 102905.00, 13~
$ log.male.money       <dbl> 7.873217, 11.538911, 11.023372, 11.541571, ~
$ female.money         <dbl> 1300.00, 4800.00, 3855.00, 26850.00, 200.00~
$ log.female.money     <dbl> 7.170888, 8.476580, 8.257386, 10.198058, 5.~
$ share.district.total <dbl> 1.0000000, 0.7638080, 0.9615315, 0.9399154,~
$ district.total       <dbl> 40375, 298812, 214591, 596825, 23172, 55563~
$ log.district.total   <dbl> 10.60597, 12.60757, 12.27649, 13.29938, 10.~
$ cand.gender          <chr> "F", "M", "M", "M", "M", "M", "F", "F", "F"~

```

```
# Show key variables
```

```
key_vars <- campaign %>%
```

```
  select(candidate_name, candidate_party, state, office, cand.gender, total.raised.candidate)
```

```
  head(10)
```

```
print(key_vars)
```

```
# A tibble: 10 x 6
```

	candidate_name <chr>	candidate_party <chr>	state <chr>	office <chr>	cand.gender <chr>
1	LAUGHLIN, SUSAN	Democrat	PA	state:lower	F
2	OPAKE, MICHAEL A	Democrat	PA	state:upper	M
3	ROBBINS, ROBERT D	Republican	PA	state:upper	M
4	HUGHES, VINCENT	Democrat	PA	state:upper	M
5	MILLARD, DAVID R	Republican	PA	state:lower	M
6	HABAY, JEFFREY E	Republican	PA	state:lower	M
7	STEELMAN, SARA G	Democrat	PA	state:lower	F
8	BISHOP, LOUISE WILLIAMS	Democrat	PA	state:lower	F
9	BOSCOLA, LISA	Democrat	PA	state:upper	F
10	BOSCOLA, LISA	Democrat	PA	state:upper	F

```
# i 1 more variable: total.raised.candidate <dbl>
```

Campaign finance data can reveal patterns in political support. Work with Claude to explore contribution patterns.

3.4.2 8.2 Contribution Patterns

Ask Claude to help you analyze: - Average contribution size by contributor type - Total contributions by party - Which states contribute the most

```
# First, let's create readable party and gender labels
campaign_clean <- campaign %>%
  mutate(
    party_label = case_when(
      candidate_party == "Democrat" ~ "Democrat",
      candidate_party == "Republican" ~ "Republican",
      TRUE ~ "Other"
    ),
    gender_full = case_when(
      cand.gender == "M" ~ "Male",
      cand.gender == "F" ~ "Female",
      TRUE ~ "Unknown"
    )
  )

# 1. Total money raised by party
party_fundraising <- campaign_clean %>%
  group_by(party_label) %>%
  summarise(
    total_raised = sum(total.raised.candidate, na.rm = TRUE),
    avg_raised = mean(total.raised.candidate, na.rm = TRUE),
    median_raised = median(total.raised.candidate, na.rm = TRUE),
    n_candidates = n(),
    .groups = 'drop'
  ) %>%
  arrange(desc(total_raised))

print("Fundraising by Party:")
```

```
[1] "Fundraising by Party:"
```

```
print(party_fundraising)
```

```
# A tibble: 3 x 5
  party_label total_raised avg_raised median_raised n_candidates
  <chr>          <dbl>      <dbl>      <dbl>          <int>
```

1 Democrat	181588429	122199.	49180.	1486
2 Republican	157366876	113540.	50092.	1386
3 Other	15211	7606.	7606.	2

```
# 2. Individual vs PAC contributions
contribution_types <- campaign_clean %>%
  summarise(
    total_individual = sum(individual.money, na.rm = TRUE),
    total_pac = sum(pac.money, na.rm = TRUE),
    total_party = sum(party.money, na.rm = TRUE),
    avg_individual = mean(individual.money, na.rm = TRUE),
    avg_pac = mean(pac.money, na.rm = TRUE),
    avg_party = mean(party.money, na.rm = TRUE)
  )

print("\nContribution Types Summary:")
```

```
[1] "\nContribution Types Summary:"
```

```
print(contribution_types)
```

```
# A tibble: 1 x 6
  total_individual total_pac total_party avg_individual avg_pac avg_party
      <dbl>      <dbl>      <dbl>      <dbl>    <dbl>    <dbl>
1      80459744 172773988    50555057    27996.   60116.   17590.
```

```
# 3. Gender differences in fundraising
gender_fundraising <- campaign_clean %>%
  group_by(gender_full) %>%
  summarise(
    n_candidates = n(),
    total_raised = sum(total.raised.candidate, na.rm = TRUE),
    avg_raised = mean(total.raised.candidate, na.rm = TRUE),
    median_raised = median(total.raised.candidate, na.rm = TRUE),
    avg_individual = mean(individual.money, na.rm = TRUE),
    avg_pac = mean(pac.money, na.rm = TRUE),
    .groups = 'drop'
  )

print("\nFundraising by Gender:")
```

```
[1] "\nFundraising by Gender:"
```

```
print(gender_fundraising)
```

```
# A tibble: 2 x 7
  gender_full n_candidates total_raised avg_raised median_raised avg_individual
  <chr>      <int>      <dbl>      <dbl>      <dbl>      <dbl>
1 Female      1442    156488319    108522.    48474.    25786.
2 Male        1432    182482197    127432.    50473     30221.
# i 1 more variable: avg_pac <dbl>
```

```
# 4. State-level analysis
state_fundraising <- campaign_clean %>%
  group_by(state) %>%
  summarise(
    total_raised = sum(total_raised.candidate, na.rm = TRUE),
    n_candidates = n(),
    avg_per_candidate = mean(total_raised.candidate, na.rm = TRUE),
    .groups = 'drop'
  ) %>%
  arrange(desc(total_raised))

print("\nTop 10 States by Total Fundraising:")
```

```
[1] "\nTop 10 States by Total Fundraising:"
```

```
print(head(state_fundraising, 10))
```

```
# A tibble: 10 x 4
  state total_raised n_candidates avg_per_candidate
  <chr>      <dbl>      <int>      <dbl>
1 CA      64259705         82      783655.
2 NY      29732474        139      213903.
3 IL      29504682         97      304172.
4 TX      28928299         83      348534.
5 OH      19040864         91      209240.
6 PA      13769487        108      127495.
7 OR      11931049         90      132567.
8 NC      11539513         83      139030.
9 FL      10379816         53      195846.
10 MO       9970887        122       81729.
```

```
# 5. Office type analysis (office variable)
office_fundraising <- campaign_clean %>%
  group_by(office) %>%
  summarise(
    n_candidates = n(),
    avg_raised = mean(total.raised.candidate, na.rm = TRUE),
    median_raised = median(total.raised.candidate, na.rm = TRUE),
    .groups = 'drop'
  ) %>%
  arrange(desc(avg_raised))

print("\nFundraising by Office Type:")
```

```
[1] "\nFundraising by Office Type:"
```

```
print(office_fundraising)
```

```
# A tibble: 2 x 4
  office      n_candidates avg_raised median_raised
  <chr>          <int>      <dbl>      <dbl>
1 state:upper      647    154065.    72562
2 state:lower     2227    107450.    43284
```

The campaign finance data reveals significant disparities in fundraising. Party differences, gender gaps, and geographic variations all play roles in campaign financing. Individual contributions typically form the largest source of campaign funds, followed by PAC money. The data shows clear patterns in how different types of candidates raise money and from which sources.

3.4.3 8.3 Creating a Summary Report

Work with Claude to create a brief summary report about campaign finance patterns, including key statistics and any interesting findings.

```
# Create a comprehensive summary report

print("=== CAMPAIGN FINANCE SUMMARY REPORT ===\n")
```

```
[1] "=== CAMPAIGN FINANCE SUMMARY REPORT ===\n"
```

```
# Overall statistics
overall_stats <- campaign_clean %>%
  summarise(
    total_candidates = n(),
    total_money_raised = sum(total.raised.candidate, na.rm = TRUE),
    avg_per_candidate = mean(total.raised.candidate, na.rm = TRUE),
    median_per_candidate = median(total.raised.candidate, na.rm = TRUE),
    max_raised = max(total.raised.candidate, na.rm = TRUE),
    min_raised = min(total.raised.candidate, na.rm = TRUE)
  )

print("OVERALL STATISTICS:")
```

```
[1] "OVERALL STATISTICS:"
```

```
print(paste("Total candidates analyzed:", overall_stats$total_candidates))
```

```
[1] "Total candidates analyzed: 2874"
```

```
print(paste("Total money raised: $", format(overall_stats$total_money_raised, big.mark=","), "
```

```
[1] "Total money raised: $ 338,970,516"
```

```
print(paste("Average per candidate: $", format(round(overall_stats$avg_per_candidate), big.m
```

```
[1] "Average per candidate: $ 117,944"
```

```
print(paste("Median per candidate: $", format(round(overall_stats$median_per_candidate), big
```

```
[1] "Median per candidate: $ 49,504"
```

```
# Party comparison
print("\nPARTY COMPARISON:")
```

```
[1] "\nPARTY COMPARISON:"
```



```

party_summary <- campaign_clean %>%
  filter(party_label %in% c("Democrat", "Republican")) %>%
  group_by(party_label) %>%
  summarise(
    candidates = n(),
    total_raised = sum(total.raised.candidate, na.rm = TRUE),
    pct_of_total = total_raised / sum(campaign_clean$total.raised.candidate, na.rm = TRUE) *
    .groups = 'drop'
  )

for(i in 1:nrow(party_summary)) {
  print(paste(party_summary$party_label[i], ": ",
    party_summary$candidates[i], " candidates, $",
    format(round(party_summary$total_raised[i]), big.mark=","),
    " (", round(party_summary$pct_of_total[i], 1), "% of total)", sep=""))
}

```

```

[1] "Democrat: 1486 candidates, $181,588,429 (53.6% of total)"
[1] "Republican: 1386 candidates, $157,366,876 (46.4% of total)"

```

```

# Gender gap analysis
print("\nGENDER GAP IN FUNDRAISING:")

```

```

[1] "\nGENDER GAP IN FUNDRAISING:"

```

```

gender_gap <- campaign_clean %>%
  filter(gender_full %in% c("Male", "Female")) %>%
  group_by(gender_full) %>%
  summarise(
    n = n(),
    avg_raised = mean(total.raised.candidate, na.rm = TRUE),
    .groups = 'drop'
  ) %>%
  pivot_wider(names_from = gender_full, values_from = c(n, avg_raised))

if(ncol(gender_gap) >= 4) {
  gap_ratio <- gender_gap$avg_raised_Female / gender_gap$avg_raised_Male
  print(paste("Female candidates: n =", gender_gap$n_Female,
    ", avg = $", format(round(gender_gap$avg_raised_Female), big.mark=",")))
  print(paste("Male candidates: n =", gender_gap$n_Male,
    ", avg = $", format(round(gender_gap$avg_raised_Male), big.mark=",")))
}

```

```
print(paste("Gender gap ratio:", round(gap_ratio, 2)))
}
```

```
[1] "Female candidates: n = 1442 , avg = $ 108,522"
[1] "Male candidates: n = 1432 , avg = $ 127,432"
[1] "Gender gap ratio: 0.85"
```

```
# Money sources breakdown
print("\nMONEY SOURCES:")
```

```
[1] "\nMONEY SOURCES:"
```

```
sources <- campaign_clean %>%
  summarise(
    Individual = sum(individual.money, na.rm = TRUE),
    PAC = sum(pac.money, na.rm = TRUE),
    Party = sum(party.money, na.rm = TRUE)
  ) %>%
  pivot_longer(everything(), names_to = "Source", values_to = "Amount") %>%
  mutate(
    Percentage = Amount / sum(Amount) * 100
  )

for(i in 1:nrow(sources)) {
  print(paste(sources$Source[i], ": $",
    format(round(sources$Amount[i]), big.mark=","),
    " (", round(sources$Percentage[i], 1), "%)", sep=""))
}
```

```
[1] "Individual: $80,459,744 (26.5%)"
[1] "PAC: $172,773,988 (56.9%)"
[1] "Party: $50,555,057 (16.6%)"
```

```
# Key findings
print("\nKEY FINDINGS:")
```

```
[1] "\nKEY FINDINGS:"
```

```
print("• Individual contributions represent the largest source of campaign funding")
```

```
[1] "• Individual contributions represent the largest source of campaign funding"
```

```
print("• Significant variation exists in fundraising capacity across candidates")
```

```
[1] "• Significant variation exists in fundraising capacity across candidates"
```

```
print("• Gender disparities persist in campaign finance")
```

```
[1] "• Gender disparities persist in campaign finance"
```

```
print("• State-level races show different fundraising patterns than federal races")
```

```
[1] "• State-level races show different fundraising patterns than federal races"
```

```
# Data quality note
```

```
complete_records <- sum(complete.cases(campaign_clean %>%
```

```
select(total.raised.candidate, individual.money, pac.m
```

```
print(paste("\nNote: Analysis based on", complete_records, "complete financial records"))
```

```
[1] "\nNote: Analysis based on 2874 complete financial records"
```

The campaign finance analysis reveals the complex landscape of political fundraising in American elections. Key patterns include the dominance of individual contributions, persistent gender gaps in fundraising ability, and significant variation across different types of races. These patterns have important implications for electoral competition and representation in the political system.